

Income Classification and Customer Segmentation for Retail Marketing

Yun Dong

May 2026

1 Overview

This project uses a census-based dataset to help a retail client better understand and prioritize potential customers. I address two related goals: identifying individuals who are more likely to belong to a higher-income group, and grouping the broader population into customer segments that are easier to understand and act on.

For the income prediction task, the final model provides a customer-level score that can be used to prioritize outreach. On the held-out test set, the model achieved a ROC-AUC[3] of 0.9540, suggesting that it is effective at ranking individuals by income likelihood. Because the higher-income group is relatively rare, I tuned the decision threshold to balance precision and recall. At the selected threshold, the model achieved a positive-class precision of 0.5947, recall of 0.6609, and F1 score of 0.6261.

For the segmentation task, I grouped individuals into five interpretable customer segments. These include working adults, children and dependents, older low-work adults, workers with investment losses, and a small high-capital-gains group. The segments help translate the data into marketing actions: broad campaigns can focus on large working-adult audiences, while more personalized premium service can be reserved for smaller high-value groups.

Together, the classification model and segmentation model give the client both an individual-level prioritization tool and a group-level marketing framework.

2 Data Understanding and Exploration

2.1 Dataset Overview

The dataset contains weighted census observations from the 1994 and 1995 Current Population Surveys. I first reviewed the raw data file and the provided column definitions to understand the structure of the dataset before modeling. The dataset contains 199,523 records and 42 columns. Conceptually, these columns fall into three groups: 40 candidate predictor variables, one sample weight column ('weight'), and one binary income label ('label'). The predictor variables include demographic attributes, employment status, industry and occupation, household structure, migration history, birthplace and citizenship, financial variables, and survey year, etc. For reproducibility, the EDA script is located at `data_exploration/eda.py`. The column-level summary generated by the script is saved as `data_exploration/column.summary.csv`, and feature distribution plots are saved under `outputs/figures/eda/`.

2.2 Feature Groups

The 40 candidate predictor variables can be broadly grouped into three feature types:

- **Numeric variables:** examples include `age`, `wage per hour`, `capital gains`, `capital losses`, `dividends from stocks`, and `weeks worked in year`.
- **String categorical variables:** examples include `education`, `marital stat`, `race`, `sex`, `tax filer stat`, `major industry code`, `major occupation code`, `household variables`, `migration variables`, and `birthplace/citizenship variables`.
- **Integer-coded categorical variables:** examples include `detailed industry recode`, `detailed occupation recode`, `num persons worked for employer`, `own business or self employed`, `veterans benefits`, and `year`. Although these variables are read as numeric by pandas, I treat them as categorical survey codes rather than continuous quantities.

In addition to these predictor columns, `weight` is read as a `float64` column and `label` is read as a string target column.

2.3 Missing Value Handling

Only one column has pandas-detected missing values: `hispanic origin`, with 874 missing values. In addition, several categorical features contain the string value `?`, which appears to represent unknown or unavailable information. For simplicity, I map both true missing categorical values and string-coded `?` values to a common `Unknown` category.

2.4 EDA Findings and Preprocessing Decisions

I also checked whether applying the sample `weight` changed the main feature distributions. I found that the weighted distributions were broadly similar to the raw record-level distributions, so this check did not change the main preprocessing decisions. Based on the EDA, I made several preprocessing decisions:

- **Survey applicability categories:** I found that many categorical features contain values such as `Not in universe` or `Not in universe or children`. I do not treat these values as ordinary missing values. Since this dataset represents the full population, these categories are informative: they often mean that a survey question was not applicable to a person, such as a child, a non-worker, or someone outside the relevant survey group.
- **Zero-inflated financial variables:** Several financial variables, such as `capital gains`, `capital losses`, and `dividends from stocks`, are highly zero-inflated and right-skewed. I treat zero as informative rather than missing, because many individuals do not have investment income or investment losses. This pattern suggests that non-zero values in these variables are rare but potentially informative signals of financial capacity. This motivates considering a tree-based model candidate, since tree models can naturally learn threshold-based patterns such as zero versus non-zero values and are less sensitive to skewed numeric distributions than purely linear models. If using a linear model or neural network, I would instead consider explicit transformations such as `log1p`, feature scaling, and zero-value indicators to make these zero-inflated and skewed variables easier to learn from.

- **Related features:** Some variables describe similar concepts at different levels of detail. For example, `detailed industry recode` and `major industry code` both describe industry. I keep both detailed and high-level versions for modeling because they may contain complementary information. I avoid manually removing one version during EDA and instead let the modeling and validation process determine whether the additional detail is useful. For business-facing interpretation, I rely more on the high-level variables because they are easier to explain to a client audience.
- **Sensitive demographic attributes:** Some demographic variables, such as `race`, `sex`, may be sensitive in a production marketing setting. I inspect them during EDA, but I would recommend fairness and compliance review before using them directly for targeting or segment definitions.
- **Target class imbalance:** The target label is highly imbalanced, with 50000+. representing only about 6.2% of records. For evaluation, I use precision, recall, F1 score, ROC-AUC, PR-AUC, and the confusion matrix in addition to accuracy, and I consider class-weighted training during model fitting.

3 Data Preprocessing

3.1 Preprocessing Rules

Data loading, cleaning, and summary tables are implemented using pandas [7].

Column type	Preprocessing decision
Multi-class categorical variables	One-hot encoded into binary indicator columns.
Binary categorical variables	Encoded as a single 0/1 indicator when appropriate, instead of creating two redundant one-hot columns.
Integer-coded categorical variables	Treated as categorical survey codes and encoded as categorical features, rather than used as continuous numeric inputs.
Continuous numeric variables	Kept as numeric inputs. For linear/PyTorch models, scaled after the train/dev/test split.
Unknown categorical values	True missing categorical values and string-coded ? values are mapped to <code>Unknown</code> .
Survey applicability categories	Values such as <code>Not in universe</code> are kept as valid categories.
Sample weight	<code>weight</code> is excluded from the feature matrix and kept separately as <code>sample_weight</code> .
Target label	50000+. is encoded as 1 and - 50000. is encoded as 0.

Table 1: Preprocessing rules used to construct the modeling dataset.

Although one-hot encoding increases the feature dimension, the categorical cardinalities in this dataset are moderate. After preprocessing, the 40 raw predictor variables expand to roughly 509 input columns, while the dataset still contains nearly 199,523 records. I therefore consider one-hot encoding manageable and appropriate for this project. For a larger production dataset with much higher-cardinality categorical variables, alternative approaches such as feature hashing, target encoding, native categorical handling, or learned embeddings could also be considered.

3.2 Data Split

I use a 70:15:15 ratio for train/validation/test split. The dataset is relatively large, with 199,523 records, so a 15% validation set still contains roughly 30,000 records, which is sufficient for stable model comparison. For simplicity and computational efficiency, I do not use technique such as full cross-validation.

3.3 Final Modeling Dataset

After preprocessing, each record is converted into a numeric feature vector. True numeric variables are kept as numeric inputs, while string categorical variables and integer-coded categorical variables are one-hot encoded. The `weight` column is excluded from the feature matrix and stored separately as `sample_weight`. The `label` column is encoded as a binary target, where 50000+. is encoded as 1 and - 50000. is encoded as 0.

For example, a raw record containing values such as `education = High school graduate`, `class of worker = Private`, and `capital gains = 0` is converted into a numeric vector containing the original numeric values plus one-hot indicators such as `education_High school graduate = 1` and `class of worker_Private = 1`. The corresponding `sample_weight` is stored separately and is not included as a model input feature.

4 Income Classification Model

4.1 Modeling Objective

The first modeling objective is to predict whether an individual belongs to the high-income group, where `label = 1` represents 50000+. and `label = 0` represents - 50000. Because the positive class only represents about 6.2% of the dataset, I treat this as an imbalanced binary classification problem. Therefore, I evaluate the model using not only accuracy, but also ROC-AUC, positive-class precision, recall, F1 score, and the confusion matrix.

4.2 Model Choice

I use XGBoost as the main model for the income classification task. This dataset contains many heterogeneous features, and it is not always obvious in advance which variables or feature interactions will be most predictive. Some features are clearly related to income, such as age, education, occupation, industry, employment status, capital gains, dividends, and weeks worked in year, while other survey-code features may still contain useful indirect signals. A tree-based boosting model is a good fit because it can automatically learn nonlinear relationships, threshold effects, and interactions among numeric and one-hot encoded categorical features. It also handles zero-inflated and skewed numeric variables, such as capital gains and dividends, without requiring feature scaling.

4.3 Hyperparameter Search and Threshold Selection

I train multiple XGBoost[2] models over a small hyperparameter grid, varying the number of trees, tree depth, learning rate, and class-imbalance weighting. For each trained model, I evaluate ROC-AUC on the validation set. Since the default classification threshold of 0.5 may be too conservative for an imbalanced dataset, I also sweep thresholds from 0.05 to 0.95 on the validation set and compute positive-class precision, recall, and F1 score at each threshold.

The final model is selected primarily by validation ROC-AUC, because ROC-AUC measures the

ranking quality of the predicted probabilities independent of a specific threshold. After selecting the model, I choose the decision threshold on the validation set to improve the positive-class F1 score. This separates model selection from threshold selection: the model is chosen for probability ranking quality, while the threshold is chosen for the desired precision-recall trade-off.

4.4 Final Model Results

The selected final model is a model with the following hyperparameters:

- `n_estimators` = 1100
- `max_depth` = 7
- `learning_rate` = 0.03
- `subsample` = 0.8
- `colsample_bytree` = 0.8
- `scale_pos_weight` = 1

The preprocessing, clustering, and evaluation metrics are implemented using scikit-learn [5]. The selected validation threshold is 0.27. On the validation set, this model achieved a ROC-AUC of 0.9562. At the selected threshold of 0.27, the positive-class precision was 0.5944, recall was 0.6731, and F1 score was 0.6313. On the held-out test set, the final model achieved a ROC-AUC of 0.9540 and an accuracy of 0.9510. For the positive class, the test precision was 0.5947, recall was 0.6609, and F1 score was 0.6261.

4.5 Confusion Matrix Interpretation

Confusion Matrix Interpretation:

	Predicted 0	Predicted 1
Actual 0	27,234	837
Actual 1	630	1,228

Table 2: Test-set confusion matrix for the final XGBoost model using the selected threshold of 0.27.

Table 2 shows the test-set confusion matrix for the final XGBoost model using the selected threshold of 0.27. The confusion matrix shows that the model captures 1,228 of the 1,858 positive-class individuals in the test set, while missing 630. It also incorrectly flags 837 negative-class individuals as positive. This confirms the trade-off introduced by lowering the decision threshold: the model identifies more likely high-income individuals, but it also sends more lower-probability individuals into the positive group.

For a retail marketing use case, this trade-off can be acceptable if the goal is to build a broader prospect list for targeted campaigns. The model should therefore be used as a ranking and prioritization tool rather than as a perfect hard classifier. Individuals with higher predicted probabilities can be prioritized for premium campaigns, while lower-score individuals can receive broader or lower-cost marketing.

5 Customer Segmentation Model

5.1 Modeling Objective

The second objective is to build interpretable customer segments for retail marketing. The goal is to group individuals with similar profiles and translate those groups into actionable marketing strategies.

5.2 Model Choice

Many unsupervised clustering methods could be used for customer segmentation, such as Kmeans[1], Gaussian Mixture Models[9], hierarchical clustering[8], or density-based clustering[6]. Due to time constraints, I use Kmeans as the primary segmentation model in this project. Kmeans is simple, scalable for this dataset size, and easy to explain in a retail marketing context.

5.3 Modeling Implementation

For the segmentation model, I use the cleaned dataset after missing-value handling rather than the fully one-hot encoded modeling dataset. This keeps the original categorical values available for profiling and makes the resulting segments easier to interpret.

I first select a subset of marketing-relevant features, including demographic variables, employment variables, household variables, education, and financial variables. I exclude the income label and sample weight from the clustering feature matrix. Numeric features are standardized using `StandardScaler`, while categorical features are converted into one-hot encoded indicators using `OneHotEncoder`. I then apply Kmeans clustering to the transformed feature matrix.

To choose the number of clusters, I compare several values of k from 3 to 6 using silhouette score and inertia. Since inertia naturally decreases as k increases, I use it as a secondary check, but I mainly choose the final number of segments based on silhouette score and whether the clusters are easy to explain for marketing purposes.

5.4 Number of Segments

I tested KMeans with $k = 3, 4, 5$, and 6. The best silhouette score was achieved at $k = 5$, so I use five clusters for the final segmentation model.

k	Silhouette score	Inertia
3	0.2299	2.0180e6
4	0.2346	1.8407e6
5	0.2520	1.6542e6
6	0.2462	1.5680e6

Table 3: Kmeans cluster selection results.

The silhouette score peaks at $k = 5$, and five segments are still simple enough to explain for marketing purposes.

5.5 Segment Interpretation

I assign descriptive segment names after reviewing the cluster profiles. The segment names are my own business labels based on the dominant patterns in each cluster. Overall, the Kmeans model

separates the population mainly by life stage, labor-force participation, and financial profile.

Cluster	Segment Name	Share	Avg. Age	Weeks Worked	High-Income Rate
0	Working Adults	47.2%	38.6	45.8	10.7%
1	Children and Dependents	29.0%	8.8	0.6	0.0%
2	Older Low-Work Adults	21.7%	59.0	2.2	1.7%
3	Workers with Investment Losses	1.9%	44.0	41.7	30.5%
4	High Capital-Gains Group	0.2%	46.8	47.4	88.3%

Table 4: Summary of the five customer segments from the Kmeans model.

- **Cluster 0: Working Adults.** This is the largest segment, representing 47.2% of the dataset. Individuals in this group are working-age adults with high labor-force participation, averaging 45.8 weeks worked per year. Their high-income rate is 10.7%, which is above the overall dataset average.
- **Cluster 1: Children and Dependents.** This segment represents 29.0% of the dataset and has an average age of only 8.8. The group has almost no labor-force participation and almost no high-income labels. This segment is best interpreted as children or dependent household members.
- **Cluster 2: Older Low-Working Adults.** This segment represents 21.7% of the dataset and has an average age of 59.0. Individuals in this group have very low wage income and few weeks worked, but some dividend income. This group likely includes older non-working, retired, or semi-retired individuals.
- **Cluster 3: Workers with Investment Losses.** This is a small segment, representing 1.9% of the dataset. Individuals in this group are working-age adults with substantial weeks worked and high capital losses. Their high-income rate is 30.5%, much higher than the dataset average.
- **Cluster 4: High Capital-Gains Group.** This is the smallest segment, representing only 0.2% of the dataset, but it has extremely high average capital gains and the highest high-income rate at 88.3%. This segment appears to represent a small but very high-value financially affluent group.

Figure 1 visualizes a random sample of 10,000 individuals from the final five-segment Kmeans model. The plot is used only as a visual aid. The clusters are generated in the full transformed feature space, not in the two-dimensional projection.

5.6 Marketing Recommendations

The segments can support two types of marketing decisions: broad campaign targeting and high-value customer prioritization.

- **Working Adults:** This is the main scalable segment for the retail client. The group is large, economically active, and works many weeks per year, with a high-income rate above the overall average. Many individuals in this segment are likely to support regular household spending but may have limited time for daily tasks. Suitable strategies include loyalty programs, household products, mid-market promotions, convenient meal solutions, grocery delivery, prepared foods, and other offers that save time or make daily life easier.

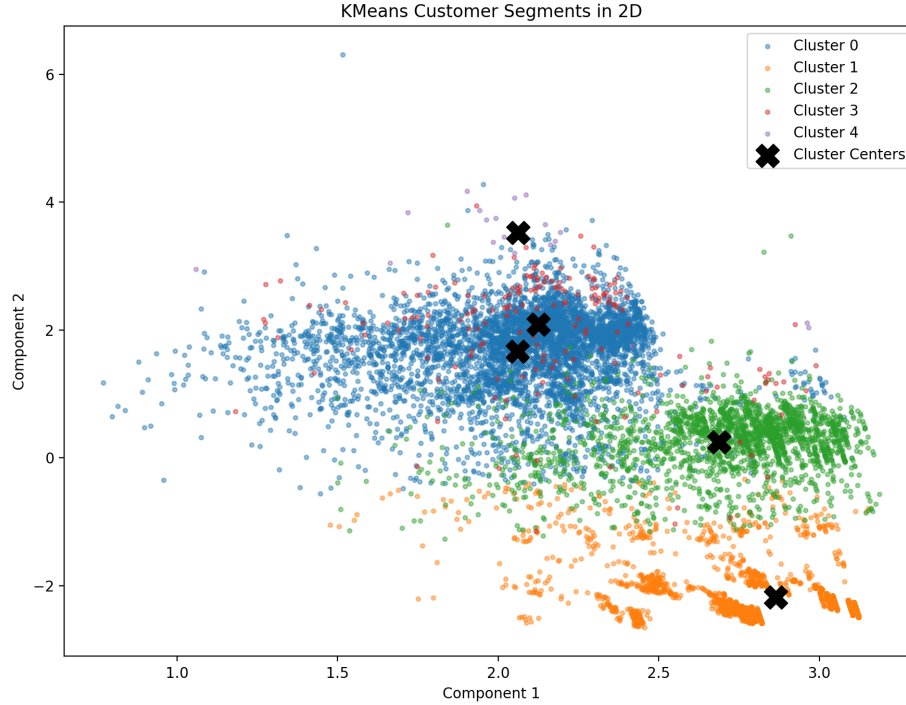


Figure 1: Two-dimensional visualization of the five KMeans customer segments. A random sample of 10,000 individuals from the full cleaned dataset is projected into two dimensions using TruncatedSVD[4] for visualization only. The Kmeans model is trained on the full transformed feature space.

- **Children and Dependents:** Do not treat this as a direct income target. Instead, use it as a household-context signal for family-oriented campaigns, such as children’s products, back-to-school promotions, and family bundles, subject to privacy and compliance constraints.
- **Older Low-Work Adults:** This group has low wage income and low labor-force participation, but may still have non-wage resources such as dividends, savings, or family support. Marketing should focus on home goods, health and wellness, senior-friendly services, value-oriented offers, and comfort-oriented products. This group may also be reached indirectly through household or family-oriented campaigns, since family members may make purchasing decisions for them.
- **Workers with Investment Losses:** This is a small but financially active segment with a much higher high-income rate than the overall population. The presence of capital losses suggests that this group may have exposure to investment or asset-market activity, so they may be more responsive to premium, differentiated, or financially sophisticated offers. For a retail client, this segment could be targeted with personalized offers, premium products, higher-margin categories, and value-added services. However, marketing should avoid overly aggressive or risk-seeking messaging and instead emphasize quality, transparency, and long-term value.
- **High Capital-Gains Group:** Treat this as a high-priority retention and relationship-management segment. Although it is very small, it has the strongest financial profile and

may represent disproportionate customer value. The client could use exclusive offers, premium products, early access programs, and high-touch personalized campaigns for this group.

6 Business Considerations, Limitations, and Future Work

Several business considerations are important when interpreting the models:

- The data comes from 1994-1995 census survey records, so demographic-income relationships may not fully represent the current retail market.
- The income label is a broad proxy for purchasing power, but it is not the same as product preference, purchase intent, or campaign responsiveness.
- The segmentation model should ideally be validated using real retail outcomes such as purchases, customer lifetime value, or campaign response.

As for the classification model: The XGBoost model provides useful ranking performance, but the positive-class precision and recall still reflect the difficulty of this imbalanced classification problem. In future work, I would compare XGBoost with additional model families, such as logistic regression, random forests, LightGBM/CatBoost, and deep neural network models.

A PyTorch-based DNN could be useful for learning nonlinear feature interactions, especially if combined with embedding layers for categorical variables instead of full one-hot encoding. Regularization methods such as dropout, weight decay, and L1 penalties could also be tested to reduce overfitting and encourage feature sparsity.

Future improvements could include:

- Incorporating actual retail transaction or campaign response data
- Evaluating fairness, privacy, and compliance considerations before production use
- Testing training and evaluation more systematically;
- Comparing additional models such as DNNs with categorical embeddings
- Validating segment stability over time
- Validating the segmentation model with real business outcomes such as purchases, customer lifetime value, or campaign response

References

- [1] Mohiuddin Ahmed, Raihan Seraj, and Syed Mohammed Shamsul Islam. The k-means algorithm: A comprehensive survey and performance evaluation. *Electronics*, 9(8):1295, 2020.
- [2] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.
- [3] Tom Fawcett. An introduction to roc analysis. *Pattern recognition letters*, 27(8):861–874, 2006.
- [4] Nathan Halko, Per-Gunnar Martinsson, and Joel A Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM review*, 53(2):217–288, 2011.

- [5] Oliver Kramer. Scikit-learn. In *Machine learning for evolution strategies*, pages 45–53. Springer, 2016.
- [6] Hans-Peter Kriegel, Peer Kröger, Jörg Sander, and Arthur Zimek. Density-based clustering. *Wiley interdisciplinary reviews: data mining and knowledge discovery*, 1(3):231–240, 2011.
- [7] Wes McKinney et al. pandas: a foundational python library for data analysis and statistics. *Python for high performance and scientific computing*, 14(9):1–9, 2011.
- [8] Fionn Murtagh and Pedro Contreras. Algorithms for hierarchical clustering: an overview. *Wiley interdisciplinary reviews: data mining and knowledge discovery*, 2(1):86–97, 2012.
- [9] Douglas A Reynolds et al. Gaussian mixture models. *Encyclopedia of biometrics*, 741(659-663):3, 2009.